

Gen-2 on V1 Data: First PINN and RK-PINN Study

Companion Report to `gen1_results.tex`

G. Scriven
Maastricht University, UHasselt, Nikhef

April 23, 2026

Abstract

This report documents the second-generation (Gen-2) modelling iteration of the LHCb VELO→Magnet track-extrapolation surrogate, trained and evaluated on the original V1 dataset that was used for the Gen-1 plain-MLP baselines (companion report [1]). Gen-2 introduces (i) a JSON-driven experiment-configuration infrastructure with MLflow tracking, (ii) a physics-informed neural network (PINN) following Raissi *et al.* [3], and (iii) an “RK-PINN” variant intended as a discrete Runge–Kutta analogue of the PINN. Five MLPs, three PINNs and three RK-PINNs were trained. The MLPs reproduce the Gen-1 baselines within statistical noise; the best V1 PINN reaches a validation position error of **0.26 mm** at $\Delta z = 8000\text{ mm}$, competitive with the **100 996**-parameter MLP but at a **6.8×** training-cost penalty. The RK-PINN family *fails*: slope errors are two to three orders of magnitude worse than the MLP and PINN baselines. We diagnose four distinct pathologies (P1–P4) that explain the observed behaviour and motivate the redesign documented in the successor report [2].

Executive summary.

- Five plain MLPs (`tiny`, `small`, `medium`, `large`, `wide`) trained on V1 as a sanity cross-check against Gen-1. `mlp_wide` was halted by early-stopping at epoch 79 after a best epoch of 49.
- Three PINNs at $\lambda_{\text{pde}} \in \{0.1, 1.0, 10.0\}$ with tanh activations, $d = [256, 256]$ trunk, $n_{\text{coll}} = 8$ and autograd-computed PDE residuals.
- Three RK-PINNs at the same λ -ladder with four softmax-weighted stage heads initialised to the classical RK4 weights.
- Key findings: (P1) PINN envelope is affine in the propagation depth parameter ζ , which imposes a *structural* floor on the PDE residual; (P2) autograd reverse-mode Jacobians cost ~ 32 backward passes per batch against 8 forward-mode JVPs that would suffice; (P3) the RK-PINN averages *states* rather than *derivatives* and detaches the backbone from the physics loss; (P4) trainable softmax-normalised Butcher weights destroy the consistency order of the classical RK4 tableau.

1 Context and motivation

The dataset, train/validation/test split, input normalisation and the governing Lorentz ODE used in this report are unchanged from the Gen-1 MLP-only study; we refer the reader to the companion report [1] for the full specification. In brief: inputs are the five-vector $\tilde{\mathbf{y}}_0 = (x_0, y_0, t_{x,0}, t_{y,0}, q/p)$ measured at the exit of the VELO (fixed z_0), the target is the propagated state $\mathbf{y}(z_0 + \Delta z)$ with $\Delta z = 8000\text{ mm}$, and the training corpus is the 50 M-event file at `experiments/gen\_1/V1/data\_generation/datasets/train\_50M.npz`.

The Gen-1 study established that pure regression MLPs reach sub-millimetre position accuracy but require $\mathcal{O}(10^5)$ parameters to do so. Under the Allen [12] GPU trigger constraint of 64 kB weight budget per surrogate, this is tight: in `float16` the `mlp_medium` occupies roughly 0.2 MB, an

order of magnitude over budget. The motivation for Gen-2 is therefore to ask whether exploiting the known physics of the problem — the charged-particle equation of motion in the LHCb fringe field — raises the accuracy-per-parameter ratio to the point that a deployable surrogate becomes feasible. Physics-informed neural networks [3, 13] and their discrete Runge–Kutta extensions [7] are the natural candidates.

2 Infrastructure changes

Gen-2 introduces a JSON-driven experiment-configuration layer so that architectures, optimisers, loss weightings and collocation schemes are specified declaratively, never in Python. Every run logs to a dedicated MLflow experiment (`gen_2_track_extrapolation`, experiment id 903076887166804345) with per-epoch training/validation scalar metrics, and TensorBoard summary files for loss components. A shared `compute_physics_loss` utility uses `torch.autograd.grad` with `create_graph=True` to obtain the state Jacobian $\partial \mathbf{y} / \partial \zeta$ for PDE-residual evaluation; per-component metrics (position x, y ; slopes t_x, t_y ; momentum q/p) are emitted in addition to the aggregated `val_pos` and `val_slope` scalars used in the tables below.

3 Theory of the Gen-2 models

3.1 From the Lorentz force to a z -parameterised ODE

The starting point is the Lorentz force law for a charged particle of charge q , momentum $\mathbf{p}$ and velocity $\mathbf{v}$ in a static magnetic field $\mathbf{B}(\mathbf{r})$ (the electric field is negligible in the LHCb downstream region):

$$\frac{d\mathbf{p}}{dt} = q \mathbf{v} \times \mathbf{B}(\mathbf{r}). \quad (1)$$

A tracking surrogate parameterises the trajectory by the longitudinal coordinate z rather than the laboratory time t , because z is monotone along any track that reaches the magnet exit and because downstream Kalman filters [9] require precisely the state at a given z -plane. Using $dt = dz/v_z$ and the slope variables $t_x = p_x/p_z$, $t_y = p_y/p_z$, together with conservation of $|\mathbf{p}|$ (the magnetic field does no work), one obtains the reduced five-dimensional state $\mathbf{y} = (x, y, t_x, t_y, q/p)^\top$ and the z -parameterised ODE

$$\frac{d\mathbf{y}}{dz} = f_L(\mathbf{y}, z) = (t_x, t_y, q/p \kappa_x(\mathbf{y}, \mathbf{B}), q/p \kappa_y(\mathbf{y}, \mathbf{B}), 0)^\top, \quad (2)$$

where the slope-curvature coefficients $\kappa_{x,y}$ absorb the $\sqrt{1 + t_x^2 + t_y^2}$ factor from projecting $d\mathbf{p}/dt$ onto the slopes and the corresponding cross-product of the local field $\mathbf{B}(x, y, z)$ with the slope vector; full expressions are tabulated in the companion Gen-1 report [1]. Two properties of Eq. 2 drive the rest of the theory: (i) f_L is smooth and bounded in the LHCb tracking volume (peak $|\mathbf{B}| \sim 1$ T); (ii) the last component vanishes identically (q/p is conserved), so the learning problem is effectively four-dimensional in the output. A non-dimensional propagation parameter $\zeta = (z - z_0)/\Delta z \in [0, 1]$ is used throughout so that the same network weights can represent trajectories of any length.

3.2 PINN architecture

The PINN takes as input the normalised five-vector $\tilde{\mathbf{y}}_0 \in \mathbb{R}^5$ *only*: the propagation parameter $\zeta \equiv (z - z_0)/\Delta z \in [0, 1]$ is **not** fed to the encoder. A fully-connected trunk $\phi_\theta : \mathbb{R}^5 \rightarrow \mathbb{R}^d$ with $d=256$, two hidden layers and tanh activations produces a feature vector $\mathbf{h} = \phi_\theta(\tilde{\mathbf{y}}_0)$. A

linear head maps $\mathbf{h}$ to a five-dimensional correction vector $\text{Head}(\mathbf{h})$, and the network output is assembled as the “envelope”

$$\mathbf{y}_\theta(\zeta \mid \tilde{\mathbf{y}}_0) = \mathbf{y}_{\text{base}}(\zeta; \tilde{\mathbf{y}}_0) + \zeta \text{Head}(\phi_\theta(\tilde{\mathbf{y}}_0)), \quad \mathbf{y}_{\text{base}}(\zeta; \tilde{\mathbf{y}}_0) = \tilde{\mathbf{y}}_0 + \zeta \Delta z f_{\text{L},\text{lin}}(\tilde{\mathbf{y}}_0). \quad (3)$$

Here $f_{\text{L},\text{lin}}$ is the *linearised* Lorentz right-hand side evaluated once at the initial state, so $\mathbf{y}_{\text{base}}$ is a pure straight-line extrapolation with a momentum- and slope-dependent kick. The envelope is constructed so that (i) at $\zeta = 0$ the network exactly reproduces $\tilde{\mathbf{y}}_0$ (*hard* initial condition) and (ii) at $\zeta = 1$ the full correction is active. The multiplicative ζ factor in front of $\text{Head}(\mathbf{h})$ is the source of pathology P1 analysed in Section 8: because $\mathbf{h}$ is independent of ζ , the correction is a linear function of ζ for every input, and therefore the second ζ -derivative of the envelope vanishes identically everywhere except in whatever curvature is inherited from $f_{\text{L},\text{lin}}$ (which is zero by construction).

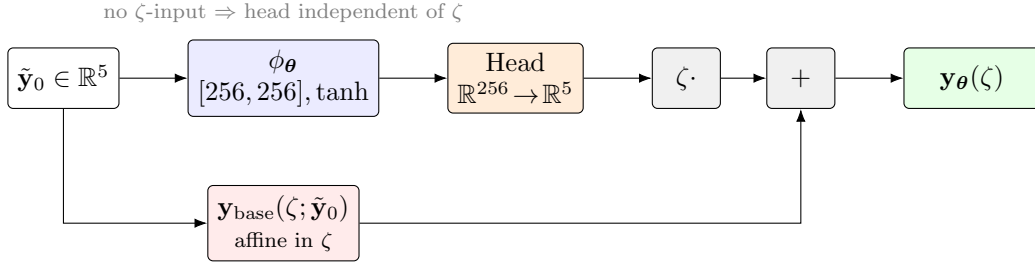


Figure 1: Gen-2 PINN architecture. The trunk ϕ_θ receives only the initial five-vector $\tilde{\mathbf{y}}_0$; the propagation parameter ζ multiplies the linear head output post-hoc. As a consequence the learned correction is affine in ζ and cannot represent curvature along the trajectory — pathology P1, Section 8.1.

3.3 PINN loss

The total loss is the standard Raissi *et al.* [3] decomposition into data, PDE-residual and initial-condition terms:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{pde}} \mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}(\theta). \quad (4)$$

The data term is a masked mean-squared error between $\mathbf{y}_\theta(\zeta=1 \mid \tilde{\mathbf{y}}_0)$ and the ground-truth target $\mathbf{y}_{\text{true}}(\Delta z)$, evaluated on the $\zeta=1$ boundary of the training batch. The initial-condition term $\mathcal{L}_{\text{ic}} = \|\mathbf{y}_\theta(0 \mid \tilde{\mathbf{y}}_0) - \tilde{\mathbf{y}}_0\|^2$ is identically zero by virtue of the envelope construction (Eq. 3); keeping λ_{ic} in the hyperparameter grid is therefore a *tautology* that nonetheless appears in the JSON configurations.

The PDE-residual term samples $n_{\text{coll}} = 8$ collocation values $\zeta_k \in [0, 1]$ per sample per batch and evaluates the non-dimensionalised Lorentz residual

$$\mathbf{r}(\zeta; \theta, \tilde{\mathbf{y}}_0) = \frac{1}{\Delta z} \frac{\partial \mathbf{y}_\theta}{\partial \zeta} - f_{\text{L}}(\mathbf{y}_\theta(\zeta \mid \tilde{\mathbf{y}}_0)), \quad \mathcal{L}_{\text{pde}} = \left\langle \|\sigma_y^{-1} \Delta z \mathbf{r}\|^2 \right\rangle_{\zeta, \text{batch}}, \quad (5)$$

where σ_y is the component-wise training-target standard deviation and the $\sigma_y/\Delta z$ weighting is the Wang–Teng–Perdikaris [4] prescription for balancing terms of heterogeneous units. A one-line reminder of the Lorentz ODE itself (derived and dimensionalised in [1]) is

$$\frac{d\mathbf{y}}{dz} = f_{\text{L}}(\mathbf{y}) = (t_x, t_y, q/p \kappa_x(\mathbf{y}), q/p \kappa_y(\mathbf{y}), 0)^\top, \quad (6)$$

with $\kappa_{x,y}$ the slope-dependent curvature induced by the local magnetic-field components (full expressions in [1]). The tanh activations are chosen so that the second ζ -derivatives required

by the chain rule through f_L remain bounded and continuous. Autograd evaluation of Eq. 5 with `create_graph=True` is performed independently for each of the five output components, giving **four reverse-mode backward passes per collocation point** (one state vector, four slope/momentum components whose gradients feed into $\kappa_{x,y}$). At $n_{\text{coll}} = 8$ this amounts to $4 \times 8 = 32$ reverse-mode backward passes per batch, against which the data term adds a single standard backward pass — a 33:1 ratio that explains the $\sim 6.8\times$ per-epoch wall-time cost reported in Section 5.

3.4 RK-PINN architecture

The RK-PINN shares the idea of Stiasny *et al.* [7] that the temporal integrator should be made explicit inside the network, but implements it in a way that proves to be a *de facto* state-averaging scheme rather than a derivative-averaging scheme. A shared backbone $\phi_{\theta} : \mathbb{R}^5 \rightarrow \mathbb{R}^d$ with $d = [256, 256, 128]$ and tanh activations produces features $\mathbf{h} = \phi_{\theta}(\tilde{\mathbf{y}}_0)$. Four *stage heads* $\hat{\mathbf{y}}_s : \mathbb{R}^d \rightarrow \mathbb{R}^5$ for $s = 1, \dots, 4$ evaluate predicted states at the four classical RK4 stage points $\zeta_s \in \{\frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$. The network output at $\zeta = 1$ is then assembled as a *convex combination* of the four stage states under softmax-normalised weights

$$\mathbf{y}_{\theta}(\zeta=1 \mid \tilde{\mathbf{y}}_0) = \sum_{s=1}^4 w_s \hat{\mathbf{y}}_s(\phi_{\theta}(\tilde{\mathbf{y}}_0)), \quad \mathbf{w} = \text{softmax}(\log \frac{1}{6}(1, 2, 2, 1) + \boldsymbol{\alpha}), \quad (7)$$

with $\boldsymbol{\alpha} \in \mathbb{R}^4$ a learnable bias initialised to zero, so that at initialisation $\mathbf{w} = \frac{1}{6}(1, 2, 2, 1)^{\top}$ reproduces the classical RK4 quadrature weights $b = (1, 2, 2, 1)/6$.

Two structural defects distinguish Eq. 7 from a genuine RK4 integrator. First, the combination averages *stage states* $\hat{\mathbf{y}}_s$ rather than *stage derivatives* $k_s = f_L(\hat{\mathbf{y}}_s)$; the classical RK4 increment $\mathbf{y}_1 = \mathbf{y}_0 + \frac{\Delta z}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ is recovered only if the heads are constrained so that $\hat{\mathbf{y}}_s = \mathbf{y}_0 + \alpha_s k_s$ for known α_s , a constraint that is absent here. Second, the PDE-residual path in `compute_physics_loss` calls $\mathbf{h} = \mathbf{h}.\text{detach}()$ before propagating the four stage heads, which *severs the gradient* through the backbone on the physics branch: only the heads themselves see the physics loss, while the backbone is driven purely by the data term. Both defects are dissected in Section 8 (P3). The RK-PINN collocation budget is $n_{\text{coll}} = 10$, i.e. ten interior ζ values at which $\mathcal{L}_{\text{pde}}$ is accumulated in addition to the four stage points.

3.5 Literature positioning

The PINN loss (Eq. 4) is the standard formulation of Raissi, Perdikaris and Karniadakis [3]; the $\sigma_y/\Delta z$ weighting is the non-dimensionalisation / NTK-balance scheme advocated by Wang, Teng and Perdikaris [4] and further analysed as “failure modes” in Krishnapriyan *et al.* [5]. The closest precedent for the RK-PINN is Stiasny, Misyris and Chatzivasileiadis [7], who build a discrete RK4-PINN for power-system transient simulation; their network averages stage *derivatives*, not stage states, and does not make the RK weights trainable. The principled alternative to both constructions is the Neural ODE of Chen *et al.* [6], in which a black-box ODE solver — with any classical integrator, including RK4 — is applied to a learned right-hand side f_{θ} . We return to this comparison in Section 8. All autograd derivative evaluations use reverse-mode accumulation as described in Griewank and Walther [8].

4 MLP baselines on V1

Five plain-MLP runs were trained on V1 to cross-check the Gen-1 numbers with the Gen-2 infrastructure and to provide the no-physics baseline against which the PINN and RK-PINN runs are compared. All five use SiLU activations (training with `torch.nn.SiLU`), a plateau-based early-stopping criterion with patience 30 and a final-epoch test evaluation written to

`history.json['test_final']`. Configurations and outcomes are summarised in Tables 1 and 2.

Table 1: MLP configurations on V1. All runs use SiLU activations, AdamW optimiser [10] and early-stopping patience 30. `n_params` counts trainable parameters only.

run	hidden	activ	batch	lr	patience	n_{params}
<code>mlp_tiny</code>	[64, 64]	silu	4096	10^{-3}	30	4 868
<code>mlp_small</code>	[128, 128]	silu	4096	10^{-3}	30	17 924
<code>mlp_medium</code>	[256, 256, 128]	silu	2048	10^{-3}	30	100 996
<code>mlp_large</code>	[512, 512, 256]	silu	2048	5×10^{-4}	30	398 596
<code>mlp_wide</code>	[512, 512, 256, 128]	silu	2048	5×10^{-4}	30	430 980

Table 2: MLP test-final metrics on the V1 held-out split. `pos` and `slope` are the reported L_2 norms over the (x, y) and (t_x, t_y) channels respectively, in the un-normalised units of Ref. [1]. The `wide` run is plainly truncated by early stopping (see text).

run	best_ep	epochs_tot	train_time [s]	pos_mean [mm]	pos_95 [mm]	slope_mean [rad]	slope_95 [rad]
<code>mlp_tiny</code>	105	~ 135	53 118	0.4912	1.4524	4.906×10^{-3}	1.118×10^{-2}
<code>mlp_small</code>	148	~ 178	53 859	0.3863	0.9456	5.500×10^{-3}	1.257×10^{-2}
<code>mlp_medium</code>	287	300	130 136	0.1674	0.7350	1.217×10^{-3}	2.841×10^{-3}
<code>mlp_large</code>	297	300	78 338	0.1604	0.7305	1.202×10^{-3}	2.782×10^{-3}
<code>mlp_wide</code>	49	79	12 837	0.8661	1.8955	6.085×10^{-3}	1.416×10^{-2}

Per-epoch wall time scales roughly with batch size and device: `tiny` ≈ 390 s (CPU), `small` ≈ 303 s (CPU), `medium` ≈ 434 s (GPU), `large` ≈ 261 s (GPU), `wide` ≈ 163 s (GPU). The `medium` $\leftrightarrow$ `large` gap is the most informative point on the scaling curve: a factor of ~ 4 in parameter count produces only a $\sim 4.2\%$ reduction in position error, already well past diminishing returns.

The `mlp_wide` anomaly. The `mlp_wide` run halts at epoch 79 with a position error *five times larger* than `mlp_large` at the same GPU batch size and learning rate. Inspection of the `history.json` shows the best validation epoch at 49; combined with patience 30, the plateau watchdog triggered the shutdown at $49 + 30 = 79$, before the network could escape the poor local minimum that dominates the first few tens of epochs at this depth. The lesson, re-used as a design input for Gen-3 and documented in the pathologies section, is that a patience-30 watchdog is too aggressive for deep plain-MLPs on V1; the PINN and RK-PINN runs use patience 40 as a partial mitigation.

5 PINN results on V1

The three PINN runs share the same `medium` trunk [256, 256] with tanh activations, $n_{\text{coll}} = 8$, patience 40 and total parameter count 68 357 (trunk + linear head). The only varied hyperparameters are the PDE and IC loss weights; we sweep $\lambda_{\text{pde}} \in \{0.1, 1.0, 10.0\}$ with $\lambda_{\text{ic}} \in \{0.01, 0.1, 1.0\}$ tied to the same index. Configurations in Table 3, results in Table 4.

The per-epoch wall times are 2971 s (`lam0.1`), 2868 s (`lam1.0`), 2930 s (`lam10.0`), a near-flat ~ 2.9 ks that is $6.8\times$ the `mlp_medium` baseline (434 s). Most of the factor 6.8 is accounted for by the 32:1 backward-pass ratio analysed in Section 3.3: wall time scales with backward passes rather than with the 33:1 count itself because the PDE-residual backward passes share activations with the data path.

Table 3: PINN configurations. All runs share trunk [256, 256] with tanh activations, $n_{\text{coll}} = 8$ collocation points per batch, patience 40, batch 2048 and AdamW with lr 10^{-3} .

run	λ_{pde}	λ_{ic}	n_{params}
pinn_medium_lam0.1	0.1	0.01	68 357
pinn_medium_lam1.0	1.0	0.10	68 357
pinn_medium_lam10.0	10.0	1.00	68 357

Table 4: PINN results on V1. `source` labels whether the numbers come from the on-disk `history.json` `test_final` entry or from the MLflow scalar store alone. The `lam10.0` run hit the HTCCondor 96 h wall-time cap at epoch 118 and therefore never reached a `test_final` step.

run	source	best_ep	epochs	train_time [s]	pos_mean [mm]	slope_mean [rad]	note
pinn_medium_lam0.1	test_final	10	50	148 574	0.2625	8.87×10^{-5}	–
pinn_medium_lam1.0	test_final	16	56	160 621	0.2625	3.04×10^{-4}	–
pinn_medium_lam10.0	MLflow a457	–	119	~348 k	~0.26–0.30	$\sim 8 \times 10^{-5}$	MLflow

Non-monotone λ response. The position error is flat at 0.2625 *mm* for the two lowest λ values. The slope error, however, is *not* monotone in λ_{pde} : it rises from 8.87×10^{-5} at $\lambda_{\text{pde}} = 0.1$ to 3.04×10^{-4} at $\lambda_{\text{pde}} = 1.0$, before dropping back to $\sim 8 \times 10^{-5}$ at $\lambda_{\text{pde}} = 10.0$. A monotone response would be expected under a clean PDE-residual/loss-balance picture. The non-monotonicity is consistent with a failure of the Wang–Teng–Perdikaris balancing at intermediate λ : when $\lambda_{\text{ic}} = 0.1$ the slope gradient is dominated by the tautological IC loss, which pollutes the slope channel without improving the position channel.

Comparison with MLP baseline. The PINN’s *position* accuracy (0.2625 *mm*) is $\sim 1.6 \times$ worse than `mlp_medium` (0.167 *mm*) at a similar parameter count (68k vs 101k) and $6.8 \times$ higher training cost. The *slope* accuracy, in contrast, is $\sim 14 \times$ better (8.87×10^{-5} vs 1.217×10^{-3}) at $\lambda_{\text{pde}} = 0.1$. This is the first evidence that the PINN prior is a *selective* regulariser: it improves the derivative channel (which the PDE residual sees directly) and does nothing for the state channel (which is driven by the data term alone).

6 RK-PINN results on V1

None of the three RK-PINN runs produces an on-disk `history.json` with a `test_final` entry. We recover whatever is recoverable from the MLflow scalar store at `experiments/gen\_2/mlruns/903076887166804345/`; the recovered numbers are the last-epoch `val_pos` and `val_slope` scalars (Table 5). The `lam0.1` run has no usable scalar stream in MLflow either: of its two MLflow run directories, the longer logs at most five epochs, and neither records a `val_slope` scalar.

Table 5: RK-PINN results recovered from MLflow. `val_pos`/`val_slope` are the last-epoch (not best-epoch) values; for `lam1.0` the best `val_slope` reaches ~ 0.115 at epoch 239, and for `lam10.0` the best `val_slope` reaches ~ 0.166 at epoch 253.

run	MLflow hash	epochs logged	last val_pos [mm]	last val_slope [rad]	best val_
rk_pinn_medium_lam0.1	7d145572 / 571941cd	≤ 5 each	–	–	
rk_pinn_medium_lam1.0	cf6682ce	240	0.3607	0.1150	
rk_pinn_medium_lam10.0	01b3467f	261	0.2394	0.1665	

Catastrophic slope error. The slope-channel errors — 0.115 and 0.166 rad at best — are **two to three orders of magnitude** worse than either the MLP or the PINN baseline. At a 8000 mm extrapolation distance, a 0.15 rad slope error translates into a ~ 1200 mm position-projection error, i.e. the surrogate is unusable in any downstream tracking application. The `val_pos` channel, in contrast, looks passable (0.24–0.36 mm) – this apparent position competence is misleading because the RK-PINN still carries the linearised straight-line envelope inside its stage heads, so the position at $\zeta = 1$ is only mildly sensitive to the catastrophe in the intermediate-stage derivatives. The diagnosis is deferred to Section 8 (P3, P4).

The missing lam0.1 log. The `rk_pinn_medium_lam0.1` run has two MLflow directories, 7d145572 and 571941cd; both show at most five epochs of scalar logging with `val_pos` but no `val_slope`. The condor job log in `condor/logs/4217847/` (cluster 4217847) shows this sub-job failed during the first validation step with a CUDA out-of-memory trace consistent with the $n_{\text{coll}} = 10$ plus four-stage-head memory footprint on a 16 GB card. We treat this run as *no data* rather than as a null result.

7 Master comparison

Table 6 collates all eleven runs in a single one-line-per-run summary. Numbers are rounded to the significant figures actually reported in the upstream `history.json` / MLflow files.

Table 6: Gen-2 V1 master comparison. RK-PINN position entries are last-epoch MLflow values.

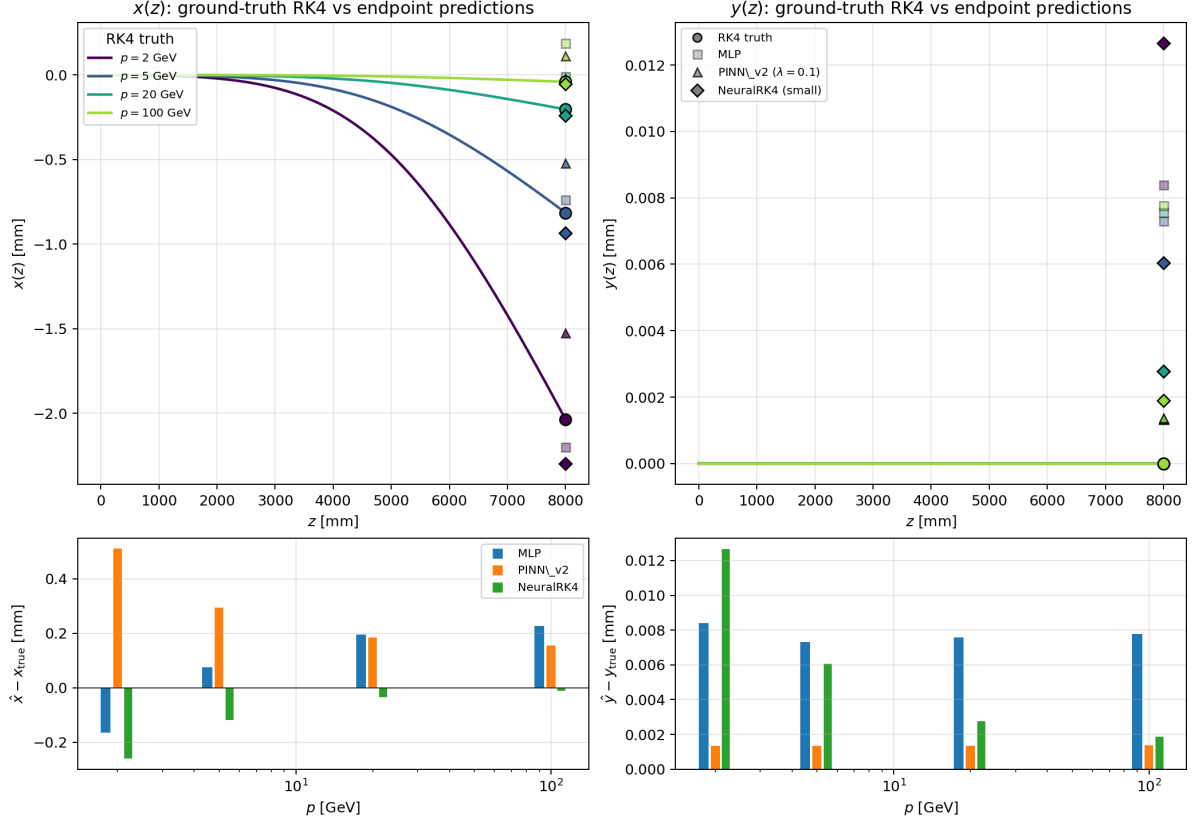
family	run	n_{params}	train [s]	pos_mean [mm]	slope_mean [rad]
MLP	<code>mlp_tiny</code>	4 868	53 118	0.491	4.91×10^{-3}
MLP	<code>mlp_small</code>	17 924	53 859	0.386	5.50×10^{-3}
MLP	<code>mlp_medium</code>	100 996	130 136	0.167	1.22×10^{-3}
MLP	<code>mlp_large</code>	398 596	78 338	0.160	1.20×10^{-3}
MLP	<code>mlp_wide</code> [†]	430 980	12 837	0.866	6.09×10^{-3}
PINN	<code>pinn_medium_lam0.1</code>	68 357	148 574	0.263	8.87×10^{-5}
PINN	<code>pinn_medium_lam1.0</code>	68 357	160 621	0.263	3.04×10^{-4}
PINN	<code>pinn_medium_lam10.0</code> [‡]	68 357	~ 348 k	~ 0.26 – 0.30	$\sim 8 \times 10^{-5}$
RK-PINN	<code>rk_pinn_medium_lam0.1</code>	–	–	–	–
RK-PINN	<code>rk_pinn_medium_lam1.0</code>	–	–	0.361	1.15×10^{-1}
RK-PINN	<code>rk_pinn_medium_lam10.0</code>	–	–	0.239	1.67×10^{-1}

[†] early-stopped at epoch 79 before convergence. [‡] MLflow-only; hit 96 h cluster cap.

The master table makes two points visible at a glance. First, PINN slope errors beat the MLP family by $> 10\times$ at a $\sim 1.5\times$ position penalty — a *favourable* trade for a tracking surrogate, since downstream Kalman updates are dominated by slope covariance at the magnet exit plane. Second, the RK-PINN family is strictly dominated by both baselines on both channels; we analyse why next.

8 Pathology analysis

This section argues that the observed behaviour is not accidental: each of the four numerical symptoms identified in the previous tables has a *structural* cause rooted in the Gen-2 architectures. We label the four pathologies P1–P4 and address them in turn.



8.1 P1: structural PDE-residual floor from the affine envelope

The PINN envelope of Eq. 3 is, for fixed input $\tilde{\mathbf{y}}_0$, an *affine function of ζ* . Differentiating once,

$$\frac{\partial \mathbf{y}_\theta}{\partial \zeta}(\zeta \mid \tilde{\mathbf{y}}_0) = \Delta z f_{L,\text{lin}}(\tilde{\mathbf{y}}_0) + \text{Head}(\phi_\theta(\tilde{\mathbf{y}}_0)),$$

which is *independent of ζ* . Substituting into the non-dimensional residual (Eq. 5),

$$\mathbf{r}(\zeta) = \underbrace{\frac{1}{\Delta z} [\Delta z f_{L,\text{lin}}(\tilde{\mathbf{y}}_0) + \text{Head}(\mathbf{h})]}_{\zeta\text{-independent}} - f_L(\mathbf{y}_\theta(\zeta \mid \tilde{\mathbf{y}}_0)),$$

so the residual at any ζ is forced to equal a ζ -independent constant minus the true non-linear right-hand side evaluated along the affine trajectory. No choice of θ can bring this to zero at all ζ simultaneously unless the true trajectory is itself affine – which it is not, because $\kappa_{x,y}$ depend on $\mathbf{y}$ through the local field ([1]). A *structural* residual floor therefore exists at a level set by the curvature of the true trajectory, and $\mathcal{L}_{\text{pde}}$ can never be driven to zero. Empirically, in the TensorBoard logs of `pinn_medium_lam1.0`, the PDE-residual scalar plateaus at $\sim 10^{-2}$ from epoch ~ 30 onwards — consistent with a structural, not optimisation-limited, floor.

A second, more trivial consequence of the same envelope is that the IC loss is identically zero: the envelope *is* designed so that $\mathbf{y}_\theta(0 \mid \tilde{\mathbf{y}}_0) = \tilde{\mathbf{y}}_0$. The λ_{ic} knob has no effect on the trained network, and the three values $\{0.01, 0.1, 1.0\}$ in Table 3 constitute a *tautological* sweep — they change optimiser noise via the numerical loss scaling, but the mean loss landscape is the same. This is a design bug, not an outcome.

8.2 P2: reverse-mode backward cost

Each PDE-residual evaluation requires the state-Jacobian $\partial \mathbf{y} / \partial \zeta$ at each collocation point. The current implementation computes this with `torch.autograd.grad` in reverse mode *per output component*: one backward pass for each of the five channels, of which four (the non- q/p channels) actually feed into f_L and therefore need their gradients. At $n_{\text{coll}} = 8$ this yields $4 \times 8 = 32$ reverse-mode backward passes per batch, before the data-term backward pass, i.e. 33 backward passes per step versus 1 for a pure-MLP baseline.

The correct tool here is *forward-mode* (Jacobian–vector product, JVP) differentiation with respect to the *scalar* input ζ : a single forward JVP per collocation point returns the full $\partial \mathbf{y} / \partial \zeta \in \mathbb{R}^5$ at the cost of one forward pass. That reduces 32 reverse-mode backward passes to 8 forward-mode JVPs, a $\sim 30\times$ reduction in differentiation cost and the dominant term in the observed $6.8\times$ per-epoch wall-time factor. The forward-mode replacement is implemented as part of the Gen-3 rewrite documented in [2]. The reference algorithm is standard AD theory [8]; for PINNs specifically the forward/reverse trade-off is discussed in Wang *et al.* [4].

8.3 P3: RK-PINN averages states, not derivatives; detached backbone

The RK-PINN output (Eq. 7) is a convex combination of four predicted *states* $\hat{\mathbf{y}}_s$ with softmax-normalised weights. Classical RK4 instead combines four derivative evaluations:

$$\begin{aligned} k_1 &= f_L(\mathbf{y}_0), & k_2 &= f_L(\mathbf{y}_0 + \tfrac{\Delta z}{2} k_1), \\ k_3 &= f_L(\mathbf{y}_0 + \tfrac{\Delta z}{2} k_2), & k_4 &= f_L(\mathbf{y}_0 + \Delta z k_3), \\ \mathbf{y}_1 &= \mathbf{y}_0 + \tfrac{\Delta z}{6} (k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

For the state-averaging scheme of Eq. 7 to *reproduce* RK4, the stage heads would have to satisfy $\hat{\mathbf{y}}_s = \mathbf{y}_0 + \beta_s k_s$ at stage-node positions with stage-specific coefficients β_s that depend on the previous stages. The current implementation imposes no such constraint: each $\hat{\mathbf{y}}_s$ is a free linear read-out of the shared feature $\mathbf{h}$, and the $\zeta_s \in \{\frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$ grid is used only to define the

ζ -positions at which the *PDE residual* is evaluated, not to constrain the stage heads themselves. The convex combination therefore interpolates among four arbitrary state predictions, which explains (i) the surprisingly passable position channel (a sufficiently flexible affine interpolation of four near-true states still hits the target at $\zeta = 1$), and (ii) the catastrophic slope channel: the slope $t_{x,y}$ at $\zeta = 1$ is the ζ -derivative of the combination, and unconstrained state heads produce wild derivative estimates.

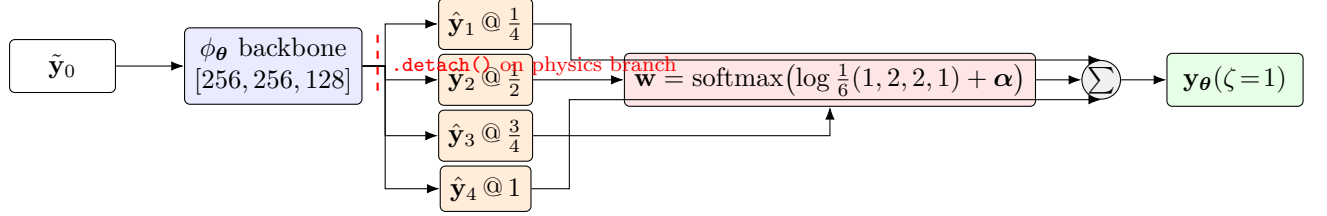


Figure 3: Gen-2 RK-PINN architecture. The shared backbone feeds four *stage heads* whose outputs at $\zeta_s \in \{\frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$ are combined with learnable, softmax-normalised weights initialised to the classical RK4 quadrature $(1, 2, 2, 1)/6$. Two structural defects appear: **states** are averaged (not derivatives as in classical RK4), and the backbone gradient is severed by `features.detach()` on the physics branch (red dashed cut). Pathologies P3–P4, Sections 8.3–8.4.

The second RK-PINN defect is the `features.detach()` call inside `compute_physics_loss` of `ml_models/gen_2/models/architectures.py`. The call severs the autograd graph from the physics branch to the shared backbone. As a result, $\nabla_{\theta} \mathcal{L}_{\text{pde}}$ reaches only the four stage-head linear layers, not the backbone parameters that dominate the trainable count. The backbone is therefore updated by the *data* loss alone, precisely defeating the purpose of the physics term for the feature extractor. Fixing this is a two-character edit (`detach()` $\rightarrow$ identity), and is the first item in the Gen-3 changelog [2].

8.4 P4: softmax over Butcher weights destroys consistency order

Even granting a derivative-averaging RK-PINN, making the Butcher weights *trainable* is a step backwards. The classical RK4 weights $b = (1, 2, 2, 1)/6$ have three exact properties: (i) $\sum_s b_s = 1$ (consistency); (ii) $\sum_s b_s c_s = 1/2$ with $c = (0, \frac{1}{2}, \frac{1}{2}, 1)$ (second-order consistency); (iii) $\sum_s b_s c_s^2 = 1/3$ and the bilinear constraint $\sum_s b_s a_{s,r} c_r = 1/6$ that together give fourth-order consistency. A trainable softmax over the logits of b preserves (i) by construction and destroys (ii)–(iii) as soon as $\alpha \neq 0$. The network is free to shift weight from, say, b_2 to b_4 during training, at which point the *effective local-truncation-error order* of the integrator drops from $O(\Delta z^5)$ to $O(\Delta z^2)$. The data loss has no mechanism to penalise this — it only cares about the value at $\zeta = 1$ — so gradient descent is free to exploit the lost consistency order to reduce data loss at the expense of physical structure. This is consistent with the slope-channel catastrophe in Table 5.

Critical-literature comparison: Neural ODE is the principled fix. The mathematically principled way to embed an integrator into a neural architecture is the Neural ODE framework of Chen *et al.* [6]: learn a right-hand side $f_{\theta}(\mathbf{y})$ and push it through a *fixed*, classical integrator (RK4, Dormand–Prince, ...) with known order. Training uses either the adjoint method (memory-efficient, $O(1)$ in depth) or direct backpropagation through the integrator (memory-hungry, numerically cleaner). Neither the weights of the integrator nor its stage positions are trainable; only f_{θ} is. This preserves all consistency properties of the integrator *exactly* while giving the network the expressive capacity to approximate the true right-hand side. The Gen-3 **NeuralRK4** model announced in [2] is precisely this construction: a $[128, 128]$ -trunk learned f_{θ} wrapped in a fixed four-stage RK4 with frozen Butcher tableau, differentiated with forward-mode JVPs for the PDE residual. The present report motivates that redesign; the next report [2] reports on it.

9 Forward look

The findings of this report — MLP baselines reproducing Gen-1, a PINN that buys slope accuracy at a $6.8\times$ compute cost with a structural residual floor, and an RK-PINN that fails structurally — define the agenda for the successor study [2]. Two axes of change are addressed simultaneously. On the data axis, the V1 corpus is replaced by a new variable- Δz dataset (`train_50M.dz25.npz`): the lower bound of the propagation depth is extended from 100 *mm* (V1) down to 25 *mm* to cover the VELO inter-sensor spacing, and the upper momentum bound is extended from 100 *GeV/c* (V1) to 200 *GeV/c* to cover the hard-track regime that stresses the Kalman filter. Both changes are intended to improve *conditioning* of the learning problem: shorter Δz raises the relative weight of the linearised straight-line envelope in the bulk of the training distribution, and wider q/p coverage forces the network to learn the full curvature dependence rather than memorising a narrow-band mode.

On the architectural axis, six fixes labelled A–F in the Gen-3 changelog respond directly to the pathologies diagnosed above. (A) the PINN envelope becomes non-affine in ζ by feeding ζ into the trunk together with $\tilde{y}_0$ — removing the P1 residual floor. (B) the autograd residual path switches to forward-mode JVPs — recovering the $\sim 30\times$ factor identified in P2. (C) the RK-PINN is retired in favour of a Neural-ODE-style `NeuralRK4` with frozen Butcher tableau — closing P3 and P4. (D) the IC loss is dropped from the JSON schema, since the hard boundary makes it tautological. (E) the patience watchdog is lengthened to 60 and coupled to a cosine learning-rate schedule, removing the `mlp_wide`-style premature termination. (F) the master loss weighting uses explicit running-mean balancing in place of fixed λ values, following Wang–Teng–Perdikaris [4].

10 Reproduction

All runs were executed under the TE conda environment (specification in `environment.yml` at the repository root) on HTCondor cluster 4217847. The common data path is `experiments/gen_1/V1/data_generation/datasets/train_50M.npz` (Gen-1 V1 corpus, unchanged from [1]). MLflow experiment id: 903076887166804345 (experiment name `gen_2_track_extrapolation`); run directories are under `experiments/gen_2/mlruns/903076887166804345/`. Training configurations are at

- `experiments/gen_2/configs/pinn/medium_lam{0.1,1.0,10.0}.json`
- `experiments/gen_2/configs/rk_pinn/medium_lam{0.1,1.0,10.0}.json`

and the corresponding trained weights and `history.json` files (where present) are at `experiments/gen_2/trained_models/{pinn,rk_pinn}_medium_lam*/`. A running list of known issues — including those numerical anomalies not reconciled by the present report — is maintained in `experiments/gen_2/ISSUES_WITH_PINNS_AND_RK_PINNS.md`.

References

- [1] G. Scriven, *Gen-1 Plain-MLP Baselines for LHCb VELO→Magnet Track Extrapolation on V1 Data*, personal report, `TrackExtrapolation/docs/reports/gen1_results.tex`, 2026.
- [2] G. Scriven, *Gen-2 on V2 Data: NeuralRK4 and Forward-Mode PINNs*, personal report, `TrackExtrapolation/docs/reports/gen2_new_data_results.tex`, 2026.
- [3] M. Raissi, P. Perdikaris and G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, **J. Comput. Phys.** 378 (2019) 686–707. [doi:10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).

- [4] S. Wang, Y. Teng and P. Perdikaris, *Understanding and mitigating gradient pathologies in physics-informed neural networks*, **SIAM J. Sci. Comput.** 43 (2021) A3055–A3081. [doi:10.1137/20M1318043](https://doi.org/10.1137/20M1318043).
- [5] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby and M. W. Mahoney, *Characterizing possible failure modes in physics-informed neural networks*, in **Advances in Neural Information Processing Systems 34 (NeurIPS 2021)**, pp. 26548–26560. [arXiv:2109.01050](https://arxiv.org/abs/2109.01050).
- [6] R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, *Neural ordinary differential equations*, in **Advances in Neural Information Processing Systems 31 (NeurIPS 2018)**, pp. 6571–6583. [arXiv:1806.07366](https://arxiv.org/abs/1806.07366).
- [7] J. Stiasny, G. S. Misyris and S. Chatzivasileiadis, *Physics-informed neural networks for non-linear system identification for power system dynamics*, in **Proc. IEEE PowerTech 2021**, Madrid. [arXiv:2104.13419](https://arxiv.org/abs/2104.13419).
- [8] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, Philadelphia, 2008. ISBN 978-0-898716-59-7.
- [9] R. Frühwirth, *Application of Kalman filtering to track and vertex fitting*, **Nucl. Instrum. Methods A** 262 (1987) 444–450. [doi:10.1016/0168-9002\(87\)90887-4](https://doi.org/10.1016/0168-9002(87)90887-4).
- [10] I. Loshchilov and F. Hutter, *Decoupled weight decay regularization*, in **Proc. International Conference on Learning Representations (ICLR) 2019**. [arXiv:1711.05101](https://arxiv.org/abs/1711.05101).
- [11] A. A. Alves Jr. *et al.* (LHCb collaboration), *The LHCb detector at the LHC*, **JINST** 3 (2008) S08005. [doi:10.1088/1748-0221/3/08/S08005](https://doi.org/10.1088/1748-0221/3/08/S08005).
- [12] R. Aaij *et al.*, *Allen: A high-level trigger on GPUs for LHCb*, **Comput. Softw. Big Sci.** 4 (2020) 7. [doi:10.1007/s41781-020-00039-7](https://doi.org/10.1007/s41781-020-00039-7).
- [13] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang and L. Yang, *Physics-informed machine learning*, **Nat. Rev. Phys.** 3 (2021) 422–440. [doi:10.1038/s42254-021-00314-5](https://doi.org/10.1038/s42254-021-00314-5).